TaxonVision-MLOps

Comprehensive Technical Implementation Guide

Systems Architecture, Infrastructure, and Codebase Reference

Benjamin Förster

October 11, 2026

Abstract

This technical report serves as an exhaustive, implementation-level architectural manual and code base reference for the TaxonVision-MLOps platform. Designed to document every structural component, configuration file, deployment manifest, mathematical loss formulation, and runtime daemon in the active repository, this document provides complete visibility into the physical server topology (`hive-mind` running Ubuntu 26.04.1 LTS, NVIDIA Driver 615.71.09, CUDA 13.4, and the NVIDIA GeForce RTX 5070 Ti with Blackwell SM 120), the lightweight Kubernetes (k3s) cluster partitioned into `jenkins-ci` and `taxon-vision` namespaces, the Cloudflare Zero Trust ingress tunnel, and the four-slot NVIDIA GPU time-slicing device plugin. Furthermore, this manual dissects the pure Pixi multi-environment toolchain, the automated DagsHub storage remote for Data Version Control (DVC) and MLflow tracking, the Cui et al. Class-Balanced Loss dynamics on biological long-tail Pareto distributions, the Optuna Bayesian hyperparameter optimisation engine, Split Conformal Prediction, Energy-Based Out-of-Distribution rejection, and the unified FastAPI and HTMX operator dashboard. Every configuration parameter, file extension, and container specification is strictly analysed based upon active codebase evidence.

Contents

Abstract **i**

1 Repository Architecture, Directory Hierarchy, and File Extension Semantics **1**
1.1 Root Directory Topology 1
1.2 Configuration Manifests and Semantics 1
1.3 Specialised File Formats and Storage Conventions 1

2 Bare-Metal Server Environment and Pixi Toolchain Interlock **2**
2.1 Physical Compute Node Baseline 2
2.2 Multi-Target Toolchain and Pixi Environment Isolation 2
2.3 Command Orchestration via Justfile 2

3 Lightweight Kubernetes (k3s) Cluster Topology and Security RBAC **3**
3.1 Cluster Architecture and Network Edge 3
3.2 Namespace Partitioning and Multi-Tenant Segregation 3
3.3 Role-Based Access Control (RBAC) Specification 3

4 GPU Virtualisation, NVIDIA Device Plugin, and VRAM Quotas **4**
4.1 The Single-GPU Multi-Tenancy Challenge 4
4.2 NVIDIA Device Plugin Time-Slicing Configuration 4
4.3 Deterministic VRAM Budget Capping and OOM Prevention 4

5 Containerisation Architecture: Dockerfile and Composition Manifests **5**
5.1 Multi-Stage Container Construction Strategy 5
5.2 Auxiliary Services and Docker Compose Topology 5
5.3 Container Hardening and Privilege Minimisation 5

6 Kubernetes Workloads, Service Manifests, and Rolling Updates **6**
6.1 API Deployment Specification 6
6.2 Health Probes and Zero-Downtime Rolling Restarts 6
6.3 Kubernetes Service Abstraction and Routing 6

7 Declarative Jenkins Pipeline and Ephemeral Kubernetes Agents **7**
7.1 Declarative Pipeline Architecture and Pod Template 7
7.2 Ephemeral Agent Lifecycle and Cache Volume Mounting 7
7.3 Stage-by-Stage Execution and Promotion Gates 7

8 Data Ingestion Engine, Zero-Copy Arrow Streaming, and License Audits **8**
8.1 iNaturalist API Ingestion and Query Mechanics 8
8.2 Zero-Copy Apache Arrow Streaming Pipeline 8
8.3 Open License Verification and Attribution Retention 8

9 Data Version Control (DVC) and Remote Storage Topology **9**
9.1 DVC Pipeline Graph and Cryptographic Manifests 9

9.2 DagsHub S3-Compatible Remote Storage Architecture 9
9.3 Synchronization Lifecycles in CI/CD and Local Workstations 9

10 Domain Modelling, DarwinCore Standards, and Taxonomic Trees 10
10.1 DarwinCore Taxon Schema and Catalog Implementation 10
10.2 Observation Entities and Spatiotemporal Metadata 10
10.3 License Verification Domain Invariants 10

11 Vision Backbone Factory and Representation Extraction Caching 11
11.1 Foundation Vision Backbone Portfolio 11
11.2 Extensible Feature Extractor Factory Architecture 11
11.3 Representation Extraction and Disk-Caching Strategy 11

12 Class-Balanced Loss Formulation and Long-Tail Gradient Dynamics 12
12.1 The Biological Long-Tail Pareto Distribution 12
12.2 Effective Number of Samples Mathematical Derivation 12
12.3 PyTorch Implementation and Numerical Stabilisation 12

13 Classification Head Architecture, Optimization Loops, and Callbacks 13
13.1 Classification Head Geometry and Transfer Learning 13
13.2 Cached Embedding Training Loop and Mini-Batch Dynamics 13
13.3 Convergence Callbacks and Best-Weight Restoration 13

14 Bayesian Hyperparameter Optimization with Optuna TPE and ASHA 14
14.1 Bayesian Optimization Principles and TPE Mechanics 14
14.2 Search Space Parameterisation and Objective Formulation 14
14.3 Asynchronous Successive Halving (ASHA) Pruning in Practice 14

15 Experiment Tracking, Cryptographic Lineage, and MLflow Model Registry 15
15.1 Experiment Tracking Architecture on DagsHub 15
15.2 Deterministic Lifecycle Management via RAII Scopes 15
15.3 Cryptographic Data-Model Lineage and Immutable Invariants 16
15.4 Metric Telemetry and Visualisation Curves 16
15.5 Artefact Persistence and MLflow Model Registry Promotion 16

16 Epistemic Safety: Split Conformal Prediction and Energy-Based OOD Rejection 18
16.1 The Epistemic Safety Problem in Biological Vision 18
16.2 Split Conformal Prediction and Statistical Coverage Bounds 18
16.3 Helmholtz Free Energy Out-of-Distribution Detection 18

17 Active Learning Strategy and Priority Triage Queue 19
17.1 Uncertainty-Based Query Acquisition Mechanics 19
17.2 Priority Triage Queue Implementation 19
17.3 Closing the Continuous Retraining Annotation Loop 19

18 ONNX Export, Zero-Allocation Runtime, and INT8 Quantization 20
18.1 Computational Graph Export and Dynamic Axes 20
18.2 Zero-Allocation C++ Inference Execution Engine 20
18.3 INT8 Quantization Calibration and Class Activation Mapping 20

19 FastAPI REST Service and Server-Rendered HTMX Operator Dashboard 21
19.1 Unified REST and Web Application Architecture 21
19.2 API Schemas, Endpoints, and Inference Routing 21
19.3 Server-Rendered HTMX Operator Interface 21

20 Real-Time Telemetry, Prometheus Instrumentation, and Drift Detection 22
20.1 Prometheus Telemetry Instrumentation and Metrics Design 22

20.2 Serving Latency and Conformal Set Monitoring 22
20.3 Embedding Representation Drift via Wasserstein and MMD 22

21 Quality Assurance: Pytest Matrix, Invariant Testing, and Complexity Budgets 23
21.1 Testing Architecture and Test Matrix Topology 23
21.2 Mathematical Invariant Assertions and Property-Based Testing 23
21.3 Cognitive Complexity Budgets and Pre-Commit Quality Gates 23

22 Open Knowledge Format (OKF v0.2) and Master Agent Routing 24
22.1 Open Knowledge Format (OKF v0.2) Schema and Trust Tiers 24
22.2 Multi-Agent Governance and Decision Routing Matrices 24
22.3 Automated Validation Scripts and D3 Knowledge Graph Visualization 24

23 Developer Operations, Environment Setup, and Engineering Runbooks 25
23.1 Developer Workstation Onboarding and Environment Setup 25
23.2 Dataset Synchronization and Remote Storage Runbook 25
23.3 Cluster Operations, GPU Debugging, and Emergency Rollback 25

Chapter 1

Repository Architecture, Directory Hierarchy, and File Extension Semantics

1.1 Root Directory Topology

1.2 Configuration Manifests and Semantics

1.3 Specialised File Formats and Storage Conventions

Chapter 2

Bare-Metal Server Environment and Pixi Toolchain Interlock

2.1 Physical Compute Node Baseline

2.2 Multi-Target Toolchain and Pixi Environment Isolation

2.3 Command Orchestration via Justfile

Chapter 3

Lightweight Kubernetes (k3s) Cluster Topology and Security RBAC

3.1 Cluster Architecture and Network Edge

3.2 Namespace Partitioning and Multi-Tenant Segregation

3.3 Role-Based Access Control (RBAC) Specification

Chapter 4

GPU Virtualisation, NVIDIA Device Plugin, and VRAM Quotas

4.1 The Single-GPU Multi-Tenancy Challenge

4.2 NVIDIA Device Plugin Time-Slicing Configuration

4.3 Deterministic VRAM Budget Capping and OOM Prevention

Chapter 5

Containerisation Architecture: Dockerfile and Composition Manifests

5.1 Multi-Stage Container Construction Strategy

5.2 Auxiliary Services and Docker Compose Topology

5.3 Container Hardening and Privilege Minimisation

Chapter 6

Kubernetes Workloads, Service Manifests, and Rolling Updates

6.1 API Deployment Specification

6.2 Health Probes and Zero-Downtime Rolling Restarts

6.3 Kubernetes Service Abstraction and Routing

Chapter 7

Declarative Jenkins Pipeline and Ephemeral Kubernetes Agents

7.1 Declarative Pipeline Architecture and Pod Template

7.2 Ephemeral Agent Lifecycle and Cache Volume Mounting

7.3 Stage-by-Stage Execution and Promotion Gates

Chapter 8

Data Ingestion Engine, Zero-Copy Arrow Streaming, and License Audits

8.1 iNaturalist API Ingestion and Query Mechanics

8.2 Zero-Copy Apache Arrow Streaming Pipeline

8.3 Open License Verification and Attribution Retention

Chapter 9

Data Version Control (DVC) and Remote Storage Topology

9.1 DVC Pipeline Graph and Cryptographic Manifests

9.2 DagsHub S3-Compatible Remote Storage Architecture

9.3 Synchronization Lifecycles in CI/CD and Local Workstations

Chapter 10

Domain Modelling, DarwinCore Standards, and Taxonomic Trees

10.1 DarwinCore Taxon Schema and Catalog Implementation

10.2 Observation Entities and Spatiotemporal Metadata

10.3 License Verification Domain Invariants

Chapter 11

Vision Backbone Factory and Representation Extraction Caching

11.1 Foundation Vision Backbone Portfolio

11.2 Extensible Feature Extractor Factory Architecture

11.3 Representation Extraction and Disk-Caching Strategy

Chapter 12

Class-Balanced Loss Formulation and Long-Tail Gradient Dynamics

12.1 The Biological Long-Tail Pareto Distribution

12.2 Effective Number of Samples Mathematical Derivation

12.3 PyTorch Implementation and Numerical Stabilisation

Chapter 13

Classification Head Architecture, Optimization Loops, and Callbacks

13.1 Classification Head Geometry and Transfer Learning

13.2 Cached Embedding Training Loop and Mini-Batch Dynamics

13.3 Convergence Callbacks and Best-Weight Restoration

Chapter 14

Bayesian Hyperparameter Optimization with Optuna TPE and ASHA

14.1 Bayesian Optimization Principles and TPE Mechanics

14.2 Search Space Parameterisation and Objective Formulation

14.3 Asynchronous Successive Halving (ASHA) Pruning in Practice

Chapter 15

Experiment Tracking, Cryptographic Lineage, and MLflow Model Registry

15.1 Experiment Tracking Architecture on DagsHub

The rigorous quantification of experimental outcomes is a foundational prerequisite for iterative model optimisation. In order to centralise experiment metadata across distributed training nodes and ephemeral continuous integration agents, the platform integrates MLflow as its primary telemetric ledger. The remote tracking server is hosted externally at `https://dagshub.com/foersben/taxon-vision-mlops.m` thereby decoupling the experiment database from local storage constraints.

Authentication against the remote DagsHub endpoint operates exclusively through non-interactive environment credentials. Within the declarative Jenkins pipeline, the `MLFLOW_TRACKING_USERNAME` and `MLFLOW_TRACKING_PASSWORD` environment variables are securely provisioned via the `DAGSHUB_TOKEN` credential. This strict non-interactive protocol ensures that CI/CD training loops proceed autonomously, while local developer environments replicate this behaviour by exporting equivalent tokens into their initialisation scripts.

Experiment naming schemes are strictly enforced via the declarative configuration schema (`settings.mlflow.experimen` This guarantees that runs are canonically grouped, preventing metadata fragmentation. Furthermore, hyperparameter optimisation phases, executed via Optuna Bayesian search, naturally generate nested run hierarchies; individual trials are registered as child runs beneath a singular parent execution, thereby preserving structural clarity within the MLflow interface.

15.2 Deterministic Lifecycle Management via RAII Scopes

The orchestration of machine learning training loops frequently encounters unpredictable termination events, such as out-of-memory anomalies or network partitions. Unhandled exceptions during these events traditionally leave orphaned, corrupted runs in the registry. To eradicate this fragility, the platform enforces Resource Acquisition Is Initialization (RAII) semantics through the `mlflow_run_scope` context manager.

This deterministic scope bounds the lifetime of an MLflow execution. Upon initialisation, the context manager actively configures the tracking URI, instantiates the run, and records the initial hyperparameter dictionaries alongside cryptographic data hashes. During execution, it yields the active run object to the underlying training loop. Should an unhandled exception propagate, the context manager intercepts the failure and safely terminates the run by mutating its status to

`FAILED` prior to re-raising the exception. In the absence of errors, execution log file artefacts are attached, and the run resolves formally as `FINISHED`.

Crucially, this architecture incorporates graceful offline degradation. If the DagsHub tracking server becomes unreachable due to a network partition, the context manager intercepts the connection fault, emits a local debug notice, and yields `None`. This guarantees that disconnected CI tests or local workstation development loops are never impeded by transient telemetry failures.

15.3 Cryptographic Data-Model Lineage and Immutable Invariants

A fundamental invariant of auditable MLOps pipelines is end-to-end traceability: every compiled model checkpoint must be cryptographically linked to the precise dataset snapshot utilised during its training. This invariant is mechanised by the `_get_dvc_dataset_hash` routine, which surgically parses the `dvc.lock` manifest prior to training. By traversing the Directed Acyclic Graph (DAG) defined within the lock file, the routine extracts the MD5 checksum corresponding to the `data/manifests` output artefact.

This dataset checksum is subsequently persisted as the `dvc_dataset_hash` parameter within the MLflow run. Concurrently, the Git commit SHA (`git.commit_sha`) and active branch identifier (`git.branch`) are extracted via shell invocation and bound as immutable run tags. Additional parameters, including the vision backbone architecture and the PyTorch per-process memory fraction (`cuda-memory-fraction`), are logged alongside these lineage markers. Consequently, any given checkpoint can be retroactively audited to determine its exact computational provenance.

15.4 Metric Telemetry and Visualisation Curves

Real-time evaluation of generalisation capacity is achieved by streaming per-epoch metrics to the remote tracking server. The `_log_mlflow_metrics` subroutine iterates over the evaluation history, sequentially emitting the training loss (`train_loss`), validation loss (`val_loss`), top-1 accuracy (`val_accuracy`), balanced F1 score (`val_f1`), and Macro PR-AUC (`val_pr_auc`) at discrete epoch steps. This chronological streaming enables the immediate visualisation of convergence trajectories and the early identification of overfitting phenomena. Furthermore, the overall training latency is quantified via the `duration_seconds` metric, allowing continuous profiling of execution efficiency across varied infrastructure tiers.

15.5 Artefact Persistence and MLflow Model Registry Promotion

Upon successful completion of the training epoch loop, physical artefacts encompassing the trained PyTorch classification head (`head.pt`), the quantised computational graph (`model.onnx`), and Optuna study parameter visualisations are serialised and persisted as MLflow artefacts.

Automated continuous deployment necessitates programmatic gating for model promotion. This gating logic is formalised within the `_promote_model_if_better` subroutine. Following artefact registration under the canonical name `taxon_vision_classifier`, the newly trained candidate is evaluated against the current production baseline. The evaluation criterion is strictly the Macro PR-AUC score (`val_promotion_score`), ensuring that generalisation across both dominant and rare taxa is preserved.

If the candidate outperforms the incumbent `Production` alias, it is automatically tagged with the `Challenger` alias, signalling its readiness for canary deployment. Conversely, inferior models are relegated to the `Archived` state. In the event that no baseline production model exists, the initial candidate is intrinsically promoted to `Production`. This mathematically rigorous promotion matrix

guarantees that algorithmic regressions are systemically blocked from entering the production serving environment.

Chapter 16

Epistemic Safety: Split Conformal Prediction and Energy-Based OOD Rejection

16.1 The Epistemic Safety Problem in Biological Vision

16.2 Split Conformal Prediction and Statistical Coverage Bounds

16.3 Helmholtz Free Energy Out-of-Distribution Detection

Chapter 17

Active Learning Strategy and Priority Triage Queue

17.1 Uncertainty-Based Query Acquisition Mechanics

17.2 Priority Triage Queue Implementation

17.3 Closing the Continuous Retraining Annotation Loop

Chapter 18

ONNX Export, Zero-Allocation Runtime, and INT8 Quantization

18.1 Computational Graph Export and Dynamic Axes

18.2 Zero-Allocation C++ Inference Execution Engine

18.3 INT8 Quantization Calibration and Class Activation Mapping

Chapter 19

FastAPI REST Service and Server-Rendered HTMX Operator Dashboard

19.1 Unified REST and Web Application Architecture

19.2 API Schemas, Endpoints, and Inference Routing

19.3 Server-Rendered HTMX Operator Interface

Chapter 20

Real-Time Telemetry, Prometheus Instrumentation, and Drift Detection

20.1 Prometheus Telemetry Instrumentation and Metrics Design

20.2 Serving Latency and Conformal Set Monitoring

20.3 Embedding Representation Drift via Wasserstein and MMD

Chapter 21

Quality Assurance: Pytest Matrix, Invariant Testing, and Complexity Budgets

21.1 Testing Architecture and Test Matrix Topology

21.2 Mathematical Invariant Assertions and Property-Based Testing

21.3 Cognitive Complexity Budgets and Pre-Commit Quality Gates

Chapter 22

Open Knowledge Format (OKF v0.2) and Master Agent Routing

22.1 Open Knowledge Format (OKF v0.2) Schema and Trust Tiers

22.2 Multi-Agent Governance and Decision Routing Matrices

22.3 Automated Validation Scripts and D3 Knowledge Graph Visualization

Chapter 23

Developer Operations, Environment Setup, and Engineering Runbooks

23.1 Developer Workstation Onboarding and Environment Setup

23.2 Dataset Synchronization and Remote Storage Runbook

23.3 Cluster Operations, GPU Debugging, and Emergency Rollback

Bibliography

[1] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.

[2] Jordan T. Ash, Chicheng Zhang, Akshay Krishnamurthy, John Langford, and Alekh Agarwal. Deep batch active learning by diverse gradient embeddings. In *International Conference on Learning Representations (ICLR)*, 2020.

[3] Yin Cui, Menglin Jia, Tsung-Yi Lin, Yang Song, and Serge Belongie. Class-balanced loss based on effective number of samples. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9268–9277. IEEE, 2019.

[4] Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Brij B. Gupta, Xiaojiang Chen, and Xinwang Wang. A survey of deep active learning. *ACM Computing Surveys (CSUR)*, 54(9):1–40, 2021.

[5] Samuel Stevens, Jiaman Wu, Matthew J. Thompson, Elizabeth G. Campolongo, Chan Hee Song, David E. Carlyn, Li Dong, Wasila M. Dahdul, Charles Stewart, Tanya Berger-Wolf, Wei-Lun Chao, and Yu Su. Bioclip: A vision foundation model for the tree of life. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19412–19424. IEEE, 2024.

[6] Grant Van Horn, Oisin Mac Aodha, Yang Song, Yin Cui, Chen Sun, Alex Shepard, Hartwig Adam, Pietro Perona, and Serge Belongie. The inaturalist species classification and detection dataset. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8769–8778. IEEE, 2018.